TACS 2026/27 · ASSIGNMENT II · KICKOFF

Agentic MDE for embedded controllers

From behavioural models to concurrent code, generated by AI agents and checked by tests

J. Pascoal Faria & Jácome Cunha · 12 November 2026

From assignment I to assignment II

ASSIGNMENT 1 · QUANTUM MDE · DONE

Structural models

- **Models:** metamodels, OCL, object models
- **Model → Model → Code:** automated transformations, by you (M2M) or by Besser (M2T)
- **Trust comes from:** the rules, applied the same way every time (like a compiler)

ASSIGNMENT 2 · AGENTIC MDE · NEXT 4 WEEKS

Behavioural models

- **Models:** state machines, sequence diagrams, timing properties
- **Model → code:** an AI agent, supervised by you
- **Trust comes from:** tests derived from the models, and your review

The models must be precise enough to guide the agent, and the tests must check that it followed them.

This week: practice session to study your problem, understand the worked example, and start the specification.

Next week: Assignment I evaluation; Assignment II dev;. In two weeks: invited talk by Abel Dantas (EU AI Act Advisory Forum).

What you build: a small embedded controller

1
Parking lot
entry and exit barriers

2
EV charging
shared power limit

3
Lift bridge
road and river traffic

4
Intersection
emergency priority

5
Home alarm
zones and keypads

6
Goods lift
two floors, one cabin

7
Tunnel
one lane, two ends

8
Level crossing
two tracks, one barrier

9
Car wash
one bay, two lanes

10
Two robots
a shared work zone

- **Two groups per problem, assigned at random:** each works independently, refines the scope, and chooses language
- **Concurrency:** two components per controller, on real threads, share a resource owned by a coordinator
- **Sensors and actuators:** the controller reads, decides and drives them, under timing constraints
- **Fake hardware:** one thread per device; real drivers would replace it behind the same message queues

What you deliver

SPEC.md

Context,
requirements spec,
design spec.
In natural language
+ **models**/diagrams
(mermaid).
About 5 to 10 pages.

TEST.md

Criteria, test scenarios with inputs and expected outcomes, how the tests are built.
Written before the code.

Code + tests

One command runs every check; the linter reports no warnings.
Folder with test scenarios (inputs and expected outcomes)

Agent files

AGENTS.md: rules for your agents.
Agent log: what each did, and what you rejected.
(Optional) skills.

Retrospective

About 2 to 5 pages, with reflections on the process.

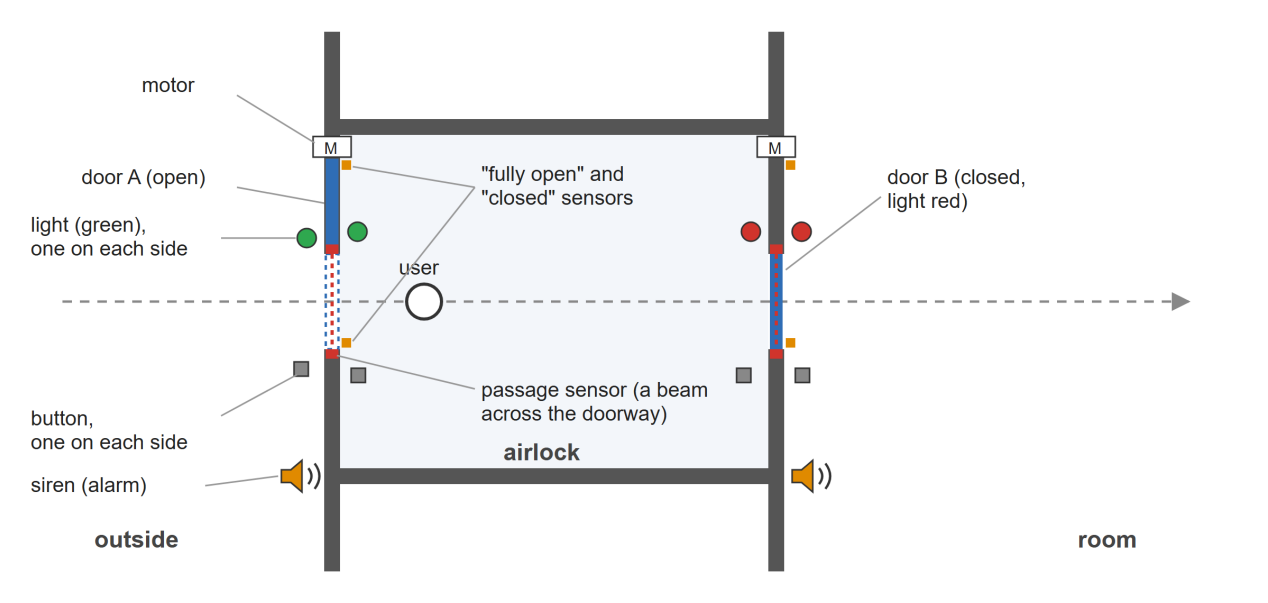
REQUIRED MODELLING

- A structural diagram (any kind)
- A state machine per concurrent component
- A sequence diagram of the main scenario

REQUIRED TESTING

- Scenarios with expected logs: every state & transition
- Every property stressed; time bounds where needed
- State machine code reviewed against its diagram

Context: The worked example: a two-door airlock



PUBLIC REPOSITORY ON GITHUB

One spec, two implementations

- SPEC.md and TEST.md shared
- rust/ and java/: the same design
- scenarios/: the same files, checked by both
- Not in the pool: copy freely, log it

- **The hazard:** both doors open at once, so the room is open to the outside
- **The design:** two door controllers, and an interlocker that holds the “token/key”, with a FIFO queue of waiting doors

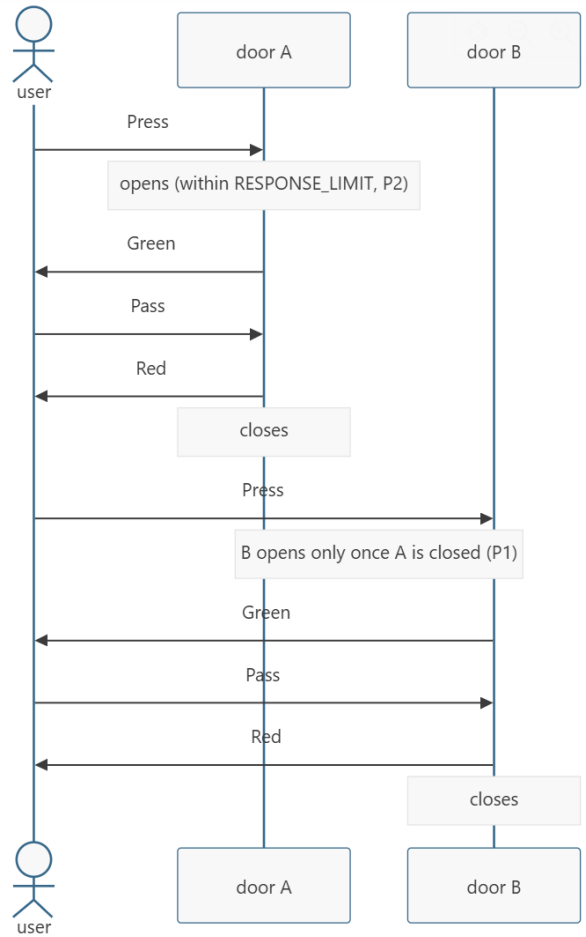
Requirements: journeys, properties & main scenario

USERS' JOURNEYS

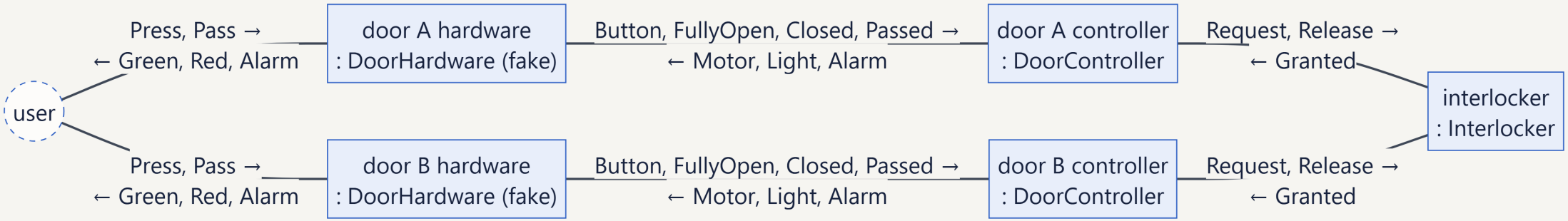
- J1: go in: A, then B
- J2: go out: B, then A
- J3: change your mind: in and back out
- J4: several users at once, at either door

Id	Property	Hazard if violated
P1	The two doors are never both not closed	the room is open to the outside
P2	A press is served within RESPONSE_LIMIT (11 s), unless an alarm comes first	a user waits for ever
P3	A movement not confirmed within MOVE_TIMEOUT (2 s) raises the alarm	a stuck door, with no warning
P4	A door is closed again if the user does not pass within MAX_OPEN (5 s)	other door blocked, airlock vulnerable

J1

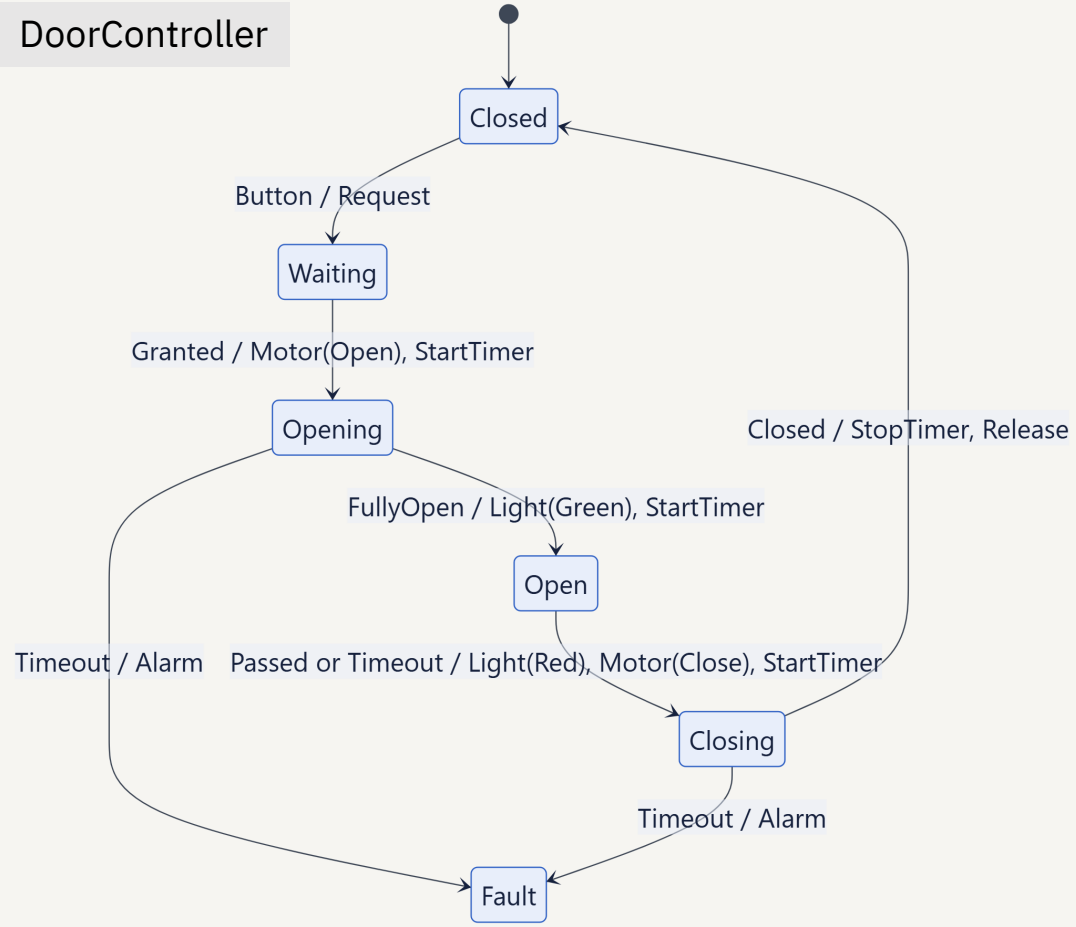


Design: active objects and messages



- **System:** each door's hardware and controller, plus an interlocker (keeping an authorization token): five **active objects** (each with its own thread); the hardware is a fake, which real drivers would replace.
- **Environment (dashed):** the users, played by the tests, who talk only to the hardware.
- **Each line:** a link; the labels are the messages, with their direction
- **One inbox per object:** a single bounded queue receives the messages of all its links. We recommend this, not a pair of queues per link: each thread waits in one place only

Design: one state machine per component



THE DECISION FUNCTION

Decides, does nothing else

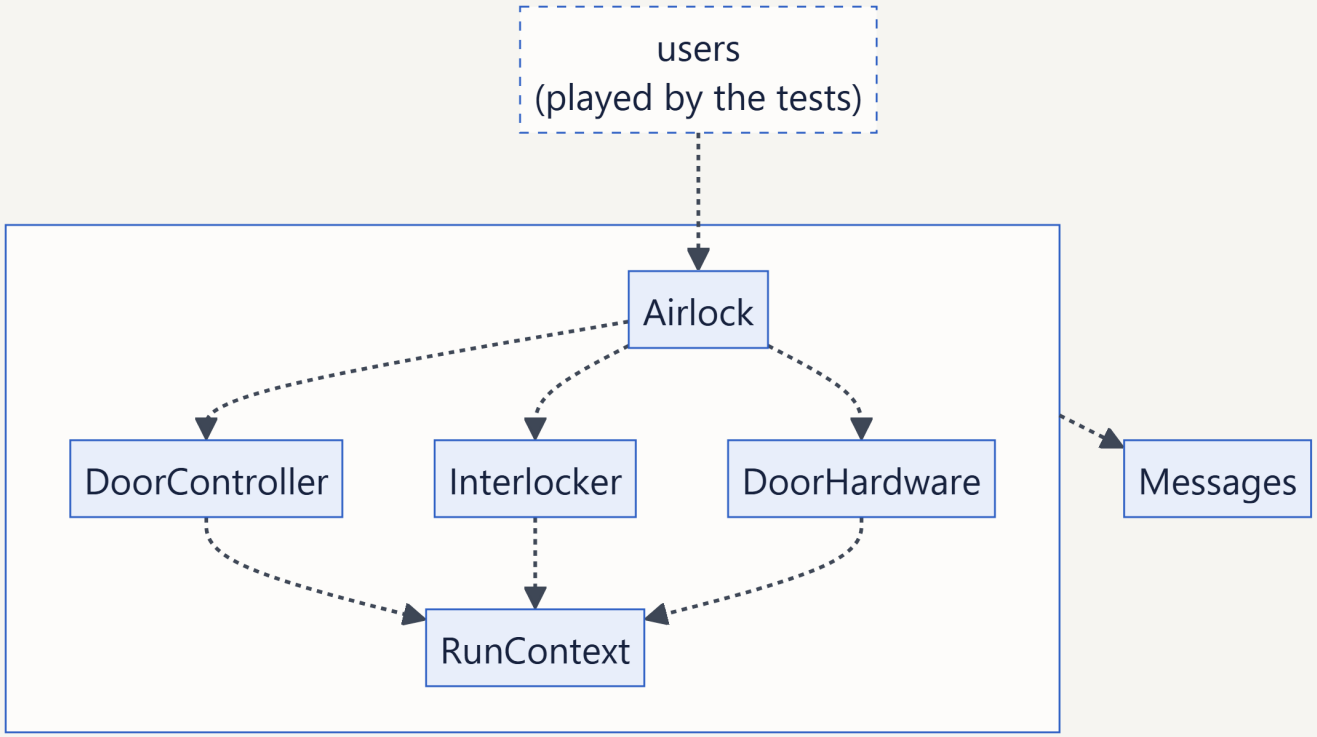
State + message → new state + actions. No I/O, threads or clock, so it can be reviewed line by line against its diagram.

SNEAK PATHS

Everything else is ignored

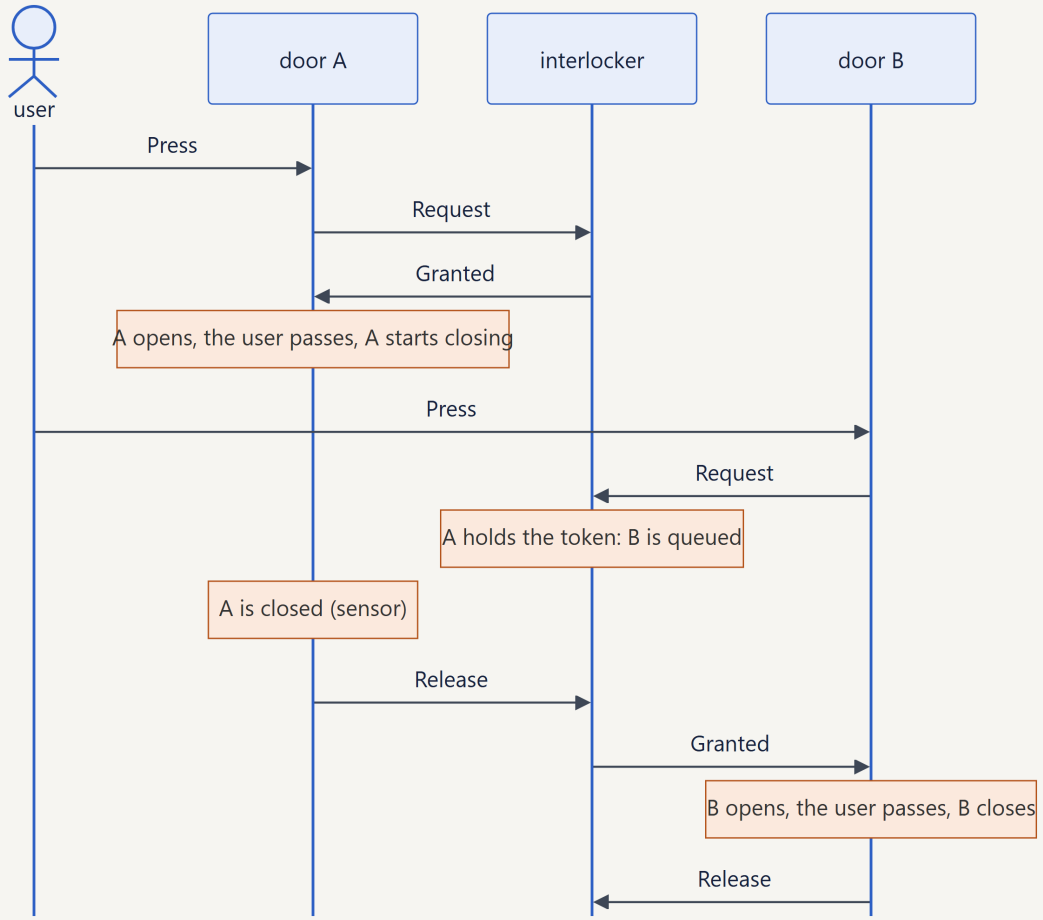
8 transitions; every other event is ignored, and the review checks it state by state. Fault is latched and keeps the token.

Design: implementation modules and dependencies



- **Airlock:** creates the queues, starts the five threads and stops them; the tests, playing the users, use only it (next slide)
- **DoorController, Interlocker, DoorHardware:** the components of the communication diagram; each runs on its own thread (the hardware is a fake driver)
- **RunContext:** what the threads of a run share: the clock, the **event log** (under a lock) and the stop flag; besides the queues, nothing else is shared
- **Messages:** the vocabulary: every message and the log entry; immutable data, used by every module

Test files: a scenario and its expected log



scenarios/s1_main.txt

```
walk = 1000
cross = 500
user 1 at 0 through a b
```

scenarios/s1_main.expected.txt (excerpt)

```
t1: In(A, Button)
Out(A, Request)
LockOut(Granted(A))
...                  # any messages
In(A, FullyOpen)     within 11000 of t1
...
LockOut(Granted(B))
```

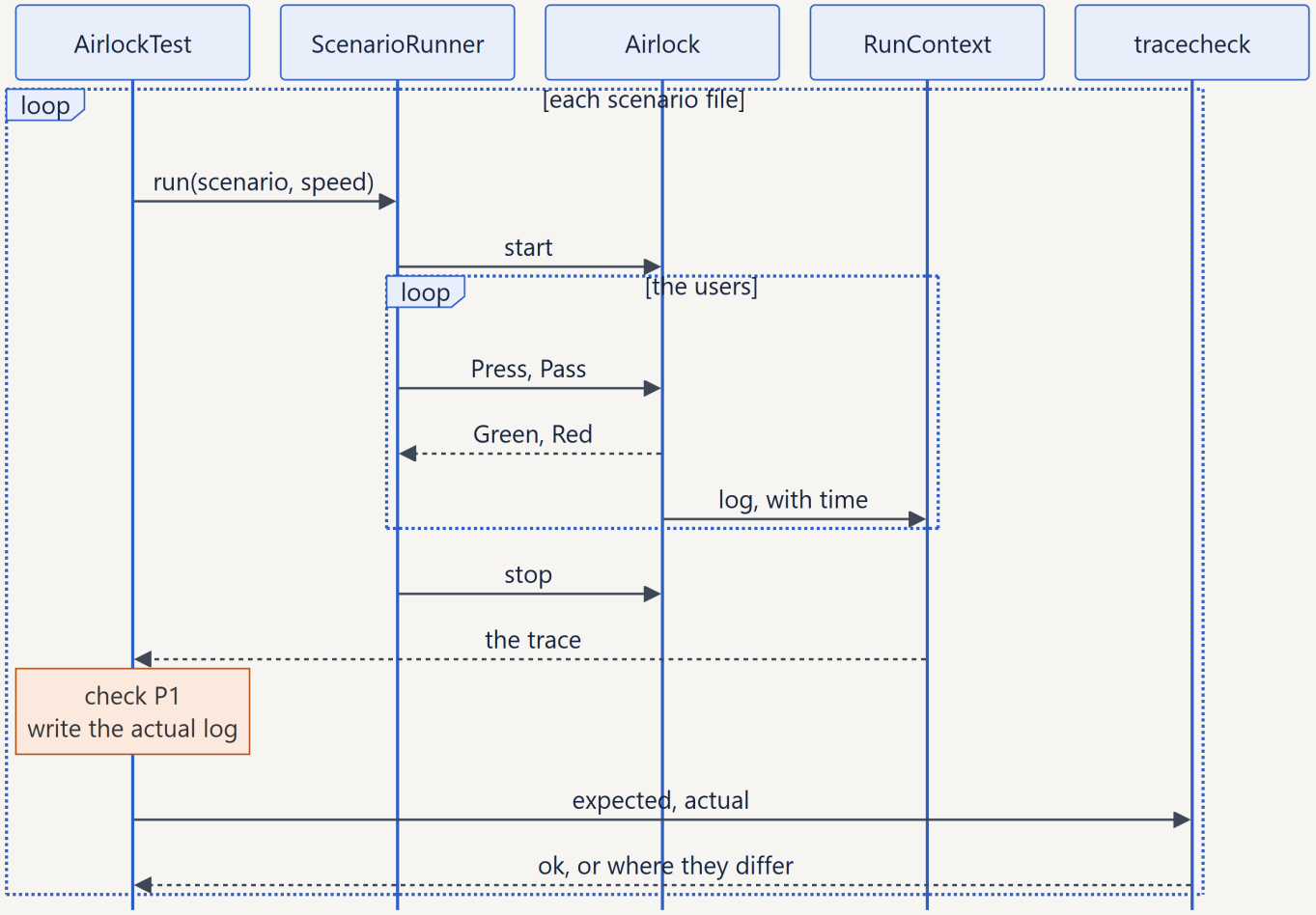
Syntax and checker tool (tracecheck) are provided to you.

Test adequacy criteria and test specs

- **Good enough when:**
  - The test scenarios cover all journeys and flows in the requirements, and all states and transitions in the design
  - All temporal properties challenged/stressed and checked in the logs
  - Scenarios allowing multiple outcomes (due to races / concurrency) are run multiple times
  - Implementation conforms to the design (state machines and messages reviewed)

Id	Scenario (TEST.md, section 2)	Takes or stresses
S1	Main scenario: in through A, then B; B pressed while A is closing	T1 to T4, T6, T9 to T12; P1, P2, P4
S2	Door A stuck while closing: the user is shut in, the alarm sounds	T8; P1, P3
S3	Door A stuck while opening	T7; P3
S4	The user is slower than MAX_OPEN: the door closes on its timer	T5; P3, P4
S5	Two users press at the same instant, one each way; run 10 times	the race for the token; P1, P2
S6	A user goes in and back out through A	a press just after closing; P2

Test architecture and flow



- **ScenarioRunner:** the test driver: starts the airlock, plays the users, stops it
- **RunContext:** every component logs each message, with its time
- **AirlockTest:** runs every scenario, checks P1 over the whole log, and writes the actual log
- **tracecheck:** the provided tool: compares the actual log with the expected one, in order and within its time bounds, in any language. Try to reuse it!

Demo: run it yourself

Rust

```
cd airlock/rust
cargo test -- --nocapture
cat target/scenarios/s1_main.actual.txt
```

Java

```
cd airlock/java
mvn test
cat target/scenarios/s1_main.actual.txt
```

What the tests print

```
s1_main: 1 runs, matched by alternative: [1]
s5_simultaneous: 10 runs, matched by alternative: [8, 2]     # either door may win the race
s6_in_and_back: 1 runs, matched by alternative: [1]
rust/target/scenarios/s1_main.actual.txt                     # the actual log, with times
```

About 30 s: the system tests run on real threads, 10 times faster than real time.

Examples of language choices

Language	Bounded queue	Tests and checks	Notes
Java 21+	ArrayBlockingQueue	JUnit; SpotBugs or Error Prone	Worked example available
Rust	sync_channel	cargo test, clippy	Worked example available. Memory and concurrency safety!
C# (.NET 8+)	BlockingCollection	xUnit or NUnit; warnings as errors	Dedicated threads, no async
Python 3.12+	queue.Queue	pytest, ruff, mypy	GIL: wider tolerance; no asyncio
C++20	mutex + condition variable	GoogleTest; ThreadSanitizer	More code; C on request

- **Pick one you can review:** the agent writes most of the code, you check every line of it
- **Real threads:** no async tasks or virtual threads, which share a thread pool

Agents need software engineering even more

CODE ONLY, PROMPT BY PROMPT

The design degrades

- **Decisions:** live in chats, and are lost between sessions
- **Each iteration:** patches the code where it fails; the agent copies what it sees
- **After a few:** nobody, human or agent, can change it safely

WITH HIGH-LEVEL ARTIFACTS

The design is kept

- **Requirements and design:** text plus precise models (state machines, properties)
- **Controlled design:** modules with one job each, explicit dependencies
- **Context in files, not chats:** kept up to date, reviewed like code

Coding agents are imperfect: good engineering matters for them as for us, and even more.

The good news: AI also helps you write and maintain these artifacts, so keeping them costs less than it used to.

In this assignment: SPEC.md, TEST.md and AGENTS.md hold the context, for people and agents alike.

Process: Specify, generate, verify, refine

STEP 1

Specify

You: requirements, design models, test plan

STEP 2

Generate

Agent: one scenario at a time, test first

STEP 3

Verify

The scenario passes; your review of the code

STEP 4

Refine

Fix the code, or the specification

↺ back to Specify when a test shows the specification was wrong

ANTI-PATTERN

Context starvation

The agent gets too little context: it guesses, and you rework. Give it SPEC.md, TEST.md and AGENTS.md.

ANTI-PATTERN

Confidence by plausibility

The code looks right, so it is accepted. Only tests and review count as evidence.

Test first, one scenario at a time

STEP 1

Add a test scenario

and its expected log, from the spec; run it, and see it **fail**

STEP 2

Make it pass

the agent implements just enough; the expected log never changes

STEP 3

Refactor

improve the code and the design, every scenario still passing; review the state machines

STEP 4

Next scenario

in the order of TEST.md: the main one, then timers, faults, several users

You command the specification (requirements, design, test plan); the agent generates the code, and may write the scenario files; you review how the specification maps to the code.

People decide, agents propose, others check



Each step reviewed by people before the next builds on it. Links by id: J1, P1, D1, S1.

People decide

requirements, design, the test oracles, what is accepted

Agents propose

drafts of documents, code, test infrastructure

Others check

review by another member; tests written independently

AGENTS.md is the team's shared prompt: the specification is the reference; if code and specification disagree, stop and say so; never change a property to make a test pass.

Agents, models and skills

THE AGENT

GitHub Copilot

free for verified students; agent mode in VS Code, the CLI, the cloud agent

THE MODEL

Auto, on the student plan

chosen for you: record the model reported for each task in the agent log

ALWAYS LOADED

AGENTS.md

the rules and the architecture, with SPEC and TEST as the reference

LOADED WHEN NEEDED

Skills

.github/skills/: the example has add-scenario, derive-expected-log, review-state-machine

Other agents (Claude Code, Cursor, Codex) are allowed, at your own cost. Check GitHub's current student offer.

NEXT STEPS

Four weeks

WEEK 1

Kickoff & specify

Problem assigned and language chosen; refine scope; install, run and explore; GitHub repo and first SPEC.md and AGENTS.md

WEEK 2 · CHECKPOINT

Main behaviour

Decision functions, first scenarios with expected logs, one command; short retrospective

WEEK 3

Concurrency

All components on threads, faults (a stuck device), stress scenarios, fixes

WEEK 4 · DELIVERY

Wrap-up

State machine review, review checklist, agent log, report, demonstration

20%	10%	20%	15%	10%	25%
Req. + design spec	Test spec	Working solution	Retrospective	Agent files + logs	Oral + progress

Product 50% (first three), process 50% (last three). Individual marks follow each student's contributions and the final discussion.

NEXT STEPS

Resources and further reading

START HERE

The assignment

- TACS_202627_Assignment2
- github.com/joaopascoalfariafeup/airlock, and its README: required vs optional

THE ONLINE BOOK

Agentic SE

- Tantithamthavorn (2026), book.agentic-swe.dev
- Ch. 6, Ch. 7.1–7.2, Tutorial 6 (~2 h)

OPTIONAL, ON THE DOMAIN

Models and testing

- Lee & Seshia, Embedded Systems (free)
- Harel (1987), Statecharts
- Binder (1999), testing state machines

ALSO WORTH WATCHING AND READING ON WORKING WITH AGENTS

- **Watch:** Matt Pocock, "Software Fundamentals Matter More Than Ever" (youtube.com/watch?v=v4F1gFy-hqg)
- **Read:** agents.md, the AGENTS.md format, read by Copilot, Codex, Cursor, Gemini CLI and others
- **Read:** martinfowler.com, "Exploring Generative AI": K. Morris, Humans and Agents in Software Engineering Loops; B. Böckeler, Context Engineering for Coding Agents, and TDD inside the agent loop

QUESTIONS

Specify precisely. Let the agent build. Trust only what you checked.

Practice session this week: problems, languages, first specification